



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 10/10

Guide Soutenance

Q/R + Demo script + Vulgarisation + Backup plan

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - Avant la soutenance	3
Chapitre 2 - Structure de la presentation	6
Chapitre 3 - Pitch 30 secondes	10
Chapitre 4 - Demo script (10 min)	12
Chapitre 5 - Vulgarisation des concepts	17
Chapitre 6 - Q/R techniques	22
Chapitre 7 - Q/R fonctionnelles	29
Chapitre 8 - Q/R pieges	34
Chapitre 9 - Backup plan	39
Chapitre 10 - Erreurs a eviter	42
Chapitre 11 - Conclusion	45

Chapitre 1 - Avant la soutenance

1.1 Checklist materiel

- + PC portable charge + chargeur
- + Cle USB avec presentation + backup PDFs
- + Adaptateur HDMI/VGA selon la salle
- + Cable RJ45 (si WiFi salle mauvaise)
- + Hotspot 4G de secours sur telephone
- + Bouteille d'eau, mouchoirs
- + Tenue professionnelle (cravate, chemise...)

1.2 Checklist technique

- + MySQL démarre + base remplie vérifiée
- + MongoDB démarre + collections OK
- + Chatbot-service tourne sur :8001
- + Academic-service tourne sur :8002
- + Frontend Angular tourne sur :4200
- + GROQ_API_KEY valide (test rapide)
- + Tester /api/health pour les 2 services
- + Browser favori avec onglets pre-ouverts
- + Screenshots/captures en backup
- + Video de demo enregistrée si tout casse

1.3 La veille

- Repeter la demo 3 fois (chronometre)
- Faire un git pull final et tester
- Mettre l'ordinateur en veille prolongee
- Imprimer 2 copies des PDFs + slides
- Dormir tot (vraiment, ca fait la difference)

1.4 Le jour J : 1h avant

- Arriver 30 min en avance

- Verifier que le retroprojecteur marche
- Demarrer tous les services + valider /health
- Faire une question test au chatbot
- Ouvrir Swagger UI dans un onglet de secours
- Respirer profondement, sourire

Chapitre 2 - Structure de la presentation

2.1 Plan classique 20 minutes

Section	Temps	Slides
1. Introduction + contexte	2 min	2
2. Problematique + objectifs	2 min	2
3. Etat de l'art	1 min	1
4. Architecture choisie	3 min	3
5. Technologies	2 min	2
6. DEMO en LIVE	5 min	0
7. Difficultes rencontrees	1 min	1
8. Resultats + chiffres	2 min	2
9. Perspectives (Phase 2)	1 min	1
10. Conclusion	1 min	1
TOTAL	20 min	15

2.2 Distribution des roles (si en binome)

Personne	Parties
Etudiant 1	Intro, contexte, problematique, demo, conclusion
Etudiant 2	Architecture, technos, difficultes, perspectives, Q/R

2.3 Slide 1 : Page de garde

Doit contenir : titre du projet, nom etudiant(s), encadrant, jury, date, logo FSBM, logo UH2C. Sobre, propre, professionnel.

2.4 Slides cle : tu DOIS avoir

- + Schema d'**architecture globale**
- + Schema du **workflow chatbot** (NLP -> reponse)
- + Diagramme du **schema BDD** (MySQL + MongoDB)
- + Comparaison **TF-IDF vs LLM**
- + Captures d'ecran **du frontend**
- + Chiffres : **nb d'intents, patterns, endpoints**
- + Captures Swagger UI

[★] REGLE D'OR : max 6 lignes par slide. Pas de phrases entieres. Des mots-cles, des images, des chiffres. Tu PARLES, le slide ILLUSTRER.

Chapitre 3 - Pitch 30 secondes

3.1 La version courte (pour debuter)

'Bonjour, nous avons developpe FSBM Platform, un assistant intelligent pour la Faculte des Sciences Ben M'Sick. Concretement, c'est un chatbot multilingue francais, anglais et darija qui repond aux questions des etudiants 24h/24 sur les inscriptions, les filieres, les emplois du temps. L'architecture est en microservices : frontend Angular, backends FastAPI, MySQL pour les donnees structurees, MongoDB pour les conversations, et LLaMA 3 pour les reponses naturelles. Aujourd'hui on va vous montrer le systeme en action.'

3.2 La version technique (pour jury exigeant)

'Nous avons concu une plateforme web modulaire repondant a 4 enjeux : disponibilite 24/7 du service scolaire, support multilingue FR/EN/Darija, comprehension du langage naturel, et scalabilite. L'architecture en microservices decouple frontend Angular 17 et 2 backends FastAPI - un pour le chatbot et un pour les donnees academiques. Le moteur NLP combine TF-IDF + cosine similarity pour la classification d'intent, avec une cascade fallback vers LLaMA 3.3-70B via Groq API enrichi par RAG. MySQL stocke les 16 tables structurees, MongoDB gere les sessions et logs. Le tout est documente via Swagger.'

3.3 Conseils delivery

- **Lent**, articule, regarde le jury
- Pas de 'euh', 'genre', 'tu vois'
- Mains visibles, pas dans les poches
- Sourire au debut + a la fin
- Si tu blanchis, prend une respiration, reprend

Chapitre 4 - Demo script (10 min)

4.1 Sequence recommandee

SCENE 1 : Accueil (1 min)

1. Ouvrir **http://localhost:4200**
 2. Montrer le dashboard avec les compteurs (25 filieres, 107 profs, 2970 etudiants)
 3. Naviguer dans le sidebar : Filieres, Departements, Annonces, Evenements
- Phrase : 'Le frontend Angular sert de hub central : un etudiant a une vue globale de la faculte.'*

SCENE 2 : Filieres (1 min)

1. Cliquer sur 'Filieres' dans sidebar
 2. Filtrer par 'MASTER'
 3. Cliquer sur 'Master IADS' -> page detail
 4. Montrer les modules par semestre, coordinateur, capacite
- Phrase : 'Les donnees viennent directement de l'academic-service via REST API. Aucune donnee codee en dur.'*

SCENE 3 : Chatbot FR (2 min)

1. Aller sur la page Chatbot
 2. Question 1 : 'Quelles sont les filieres ?' -> reponse en FR
 3. Montrer le badge 'intent: filieres, confidence: 1.0'
 4. Question 2 : 'Conditions inscription Master IADS ?' -> reponse detaillee
 5. Question 3 : 'Comment t'appelles-tu ?' -> presentation perso
- Phrase : 'Le bot detecte la langue, classifie l'intent en 30ms, repond.'*

SCENE 4 : Chatbot EN (1 min)

1. 'What are the masters in computer science ?'
 2. Reponse en anglais natif
 3. Montrer le badge 'language: en'
- Phrase : 'La detection de langue est hybride : script, mots-cles, et bascule auto'*

SCENE 5 : Chatbot Darija (2 min)

1. 'salam, ki dayer ?' -> salutation darija
 2. 'shno hia les filieres ?' -> filieres en darija
 3. 'ana 7ass b stress f les examens' -> reponse motivante darija
 4. Montrer comment le mot 'ana' detecte un user feminin si nom de fille suit
- Phrase : 'Darija est rare dans les chatbots. On a fait un dataset de 461 patterns et un algo de detection numerique 3/7/9 typique.'*

SCENE 6 : LLM Mode (2 min)

1. Switch toggle 'LLM Mode'
2. Question complexe : 'Compare les masters IADS et MIAGE pour mon profil math.'
3. Reponse longue, contextualisee, nuancee
4. Montrer le badge 'provider: groq, model: llama-3.3-70b'
5. Montrer 'contexts_used' (le RAG)

Phrase : 'Quand TF-IDF ne suffit pas, on bascule sur LLaMA 3 avec RAG. Le LLM utilise notre dataset comme contexte, donc pas d'hallucination.'

SCENE 7 : Swagger (1 min)

1. Ouvrir <http://localhost:8001/docs>
2. Montrer la liste d'endpoints
3. Click 'Try it out' sur POST /api/chat
4. Executer en live

Phrase : 'FastAPI genere automatiquement cette doc interactive. Tres utile en developpement et pour les autres devs.'

SCENE 8 : BDD (1 min)

1. Ouvrir MySQL Workbench
2. Montrer la table 'conversations' -> la derniere conv en bas
3. Ouvrir MongoDB Compass
4. Montrer la collection 'sessions' avec le tableau messages

Phrase : 'Chaque interaction est tracee : MySQL pour les stats, MongoDB pour la session complete.'

Chapitre 5 - Vulgarisation des concepts

Le jury n'est pas forcément spécialiste de chaque techno. Il faut SAVOIR EXPLIQUER SIMPLEMENT.

5.1 'C'est quoi un microservice ?'

'Imaginez un restaurant. L'approche classique = 1 cuisinier qui fait TOUT (entree, plat, dessert). Si il est malade, le restaurant ferme. Les microservices = plusieurs cuisiniers specialistes. L'un fait les entrees, l'autre les desserts. Si l'un est malade, les autres continuent. Notre projet a 2 services : un pour le chatbot, un pour les donnees academiques.'

5.2 'C'est quoi TF-IDF ?'

'TF-IDF c'est une facon de transformer du texte en chiffres. Imaginez que vous comparez 2 livres : si le mot ETUDIANT apparait dans tous les livres, il ne distingue rien. Mais ALGORITHMIQUE apparait surtout dans un livre informatique, donc c'est un indicateur fort. TF-IDF donne plus de poids aux mots distinctifs. On compare la question de l'user a TOUS nos patterns, et on choisit le plus similaire.'

5.3 'C'est quoi un LLM ?'

'Un LLM, comme LLaMA 3 ou GPT, c'est un modele entraine sur des MILLIARDS de mots. Il a appris la grammaire, les concepts, les patterns. Quand on lui pose une question, il genere une reponse mot par mot, en choisissant le plus probable a chaque etape. 70 milliards de parametres dans LLaMA 3.3, ce qui le rend tres performant.'

5.4 'C'est quoi RAG ?'

'RAG = Retrieval Augmented Generation. Probleme : les LLM peuvent INVENTER. Solution RAG = AVANT de demander au LLM, on cherche dans notre base de donnees les infos pertinentes, et on les colle dans le prompt. Le LLM REFORMULE ces vrais infos au lieu d'inventer. Comme un eleve qui repond a partir de ses notes au lieu de bluffer.'

5.5 'Pourquoi MySQL ET MongoDB ?'

'MySQL = relationnel, parfait pour les donnees structurees rigides (etudiants, filieres, modules). Les relations sont strictes. MongoDB = NoSQL, parfait pour les donnees flexibles (conversations dont la longueur varie). Mettre tout dans MySQL = lent et rigide. Mettre tout dans MongoDB = perte des relations. On combine les forces des deux : polyglot persistence.'

5.6 'C'est quoi FastAPI ?'

'FastAPI est le framework Python le plus moderne pour creer des APIs. Il est async, donc tres rapide (10000+ req/sec). Il auto-documente via Swagger. Il valide les inputs automatiquement avec Pydantic. C'est devenu un standard pour les nouveaux projets, remplaçant Flask et Django REST.'

5.7 'Pourquoi Angular 17 et pas React ?'

'Angular est un framework complet (routing, forms, HTTP), pret a l'emploi. C'est TypeScript native, donc moins de bugs. Angular 17 a introduit les SIGNALS, plus performants que le change detection classique. C'est aussi notre choix academique : plus structurant pour un projet de cette taille.'

Chapitre 6 - Q/R techniques

Q : Pourquoi async/await en Python ?

R : Le serveur peut traiter plusieurs requetes en parallele sans creer 1 thread par requete. C'est crucial pour les operations I/O (DB, HTTP). Un endpoint async libere le thread pendant l'attente DB et peut servir un autre user.

Q : Comment garantir la coherence des donnees entre MySQL et MongoDB ?

R : Phase 1 : on accepte une coherence finale eventuelle. Phase 2 prevue : transactions distribuees via outbox pattern ou Saga. Aujourd'hui, MongoDB est append-only pour les conversations, donc pas de conflit.

Q : Comment gerez-vous la scalabilite ?

R : Architecture stateless permet le scale horizontal (lancer 10 instances de chatbot-service derriere un load balancer). MySQL : read replicas. MongoDB : sharding. NLP : cache TF-IDF en RAM, partage entre instances.

Q : Quelle est la latence moyenne ?

R : TF-IDF : ~150 ms p50, 200 ms p95. LLM Groq : ~250 ms p50, 600 ms p95. Sans cache. Avec cache Redis (Phase 2), on viserait 50 ms pour les reponses repetees.

Q : Comment gerez-vous les pannes du Groq API ?

R : Cascade fallback : Groq -> HuggingFace -> TF-IDF local. Donc 100% de disponibilite garantie meme si Groq est down. Les utilisateurs ont une reponse, juste moins naturelle si on est en mode TF-IDF.

Q : Pourquoi choisir LLaMA et pas GPT-4 ?

R : Trois raisons : (1) Groq offre LLaMA gratuitement, GPT-4 coute, (2) LLaMA 3.3-70B tourne sur Groq LPU a 200+ tokens/sec vs 50 pour GPT-4, (3) Open source = deployable on-premise plus tard si besoin de confidentialite.

Q : Comment validez-vous la qualite des reponses ?

R : Trois mecanismes : (1) Confidence score TF-IDF visible dans les reponses, (2) Feedback user (1-5 etoiles) stocke pour analyse, (3) Tests automatises sur 50 questions de reference. On vise un taux 'is_helpful' > 80%.

Q : Quelle est la couverture de tests ?

R : Aujourd'hui, tests manuels via Swagger + 50 questions canoniques. Pour Phase 2 : tests unitaires pytest sur classifier, integration tests sur endpoints, smoke tests E2E.

Q : Comment evitez-vous le SQL injection ?

R : On utilise EXCLUSIVEMENT SQLAlchemy avec requetes parametrees. Pas de concatenation manuelle de SQL. Pydantic valide les inputs avant. Donc immunise.

Q : Pourquoi avoir code la detection de genre ?

R : Pour la personnalisation darija. En darija, 'khoya' (mon frere) vs 'khti' (ma soeur) vs 'lalla' (madame) sont importants. On detecte le genre via prenom + mots-cles linguistiques ('ana mra' = je suis une femme). Le bot adapte son vocabulaire.

Q : Vous avez utilise des modeles de ML pretraines ?

R : Pour le NLP : pas de pretraines, on a entraine TF-IDF sur notre dataset (28 intents, ~835 patterns). Pour le LLM : oui, LLaMA 3.3 et Mistral pretraines. C'est un mix intentionnel : leger local + puissant cloud.

Q : Comment garantir la confidentialite des messages user ?

R : Phase 1 : pas de personnal data dans les conversations (pas de mots de passe, etc.). Phase 2 : JWT pour auth, HTTPS obligatoire, RBAC pour acces aux logs. Anonymisation des analytics.

Chapitre 7 - Q/R fonctionnelles

Q : Quel est l'apport pour la FSBM ?

R : Le service scolarite a une charge enorme : 2970 etudiants, ~50 questions/jour. Le bot repond 24/7 sur les 80% de questions repetitives (filières, inscriptions, emplois du temps). La scolarite peut se concentrer sur les cas complexes.

Q : Combien d'utilisateurs peut-il supporter ?

R : Sur 1 serveur 4 cores 8GB RAM : ~200 reqs/sec simultanees. Avec scale horizontal, on peut monter a 10000+ reqs/sec. La FSBM compte ~3000 etudiants, donc largement couvert.

Q : Comment les etudiants y accedent ?

R : Phase 1 : web app responsive. Phase 2 : integration au site fsbm.ma + Messenger Facebook + WhatsApp Business API. Phase 3 : app mobile native (Flutter).

Q : Comment le contenu reste a jour ?

R : Le dataset est en JSON modifiable. Phase 2 : interface admin pour scolarite, qui peut ajouter des FAQs sans toucher au code. Auto-rechargement du modele NLP.

Q : Le chatbot peut-il aider au cas par cas ?

R : Pour les questions genrales : oui. Pour les cas personnels (releves, notes, etc.) : Phase 2 prevue avec auth etudiant -> acces a son dossier personel via JWT.

Q : Comment validez-vous le contenu ?

R : Le dataset a ete construit a partir des FAQ officielles + entretiens avec la scolarite. Le Google Form distribue a permis de collecter les questions reelles des etudiants.

Q : Que se passe-t-il si le bot ne comprend pas ?

R : Confidence < 0.3 : on retourne 'Je ne suis pas sur de comprendre, peux-tu reformuler ?' + suggestions de questions proches. On loggue ces cas pour ameliorer le dataset.

Q : Et si l'etudiant pose une question hors scope (ex: politique) ?

R : Le bot detecte une faible confiance et redirige vers les sujets supportes. Il ne repond JAMAIS sur des sujets politiques ou personnels. C'est garanti par notre dataset borne et le system prompt en mode LLM.

Q : Avez-vous des metriques d'utilisation ?

R : MongoDB analytics : count par intent, par langue, par jour. MySQL : conversations log + feedback. Tableau de bord Phase 2 prevu pour la scolarite.

Q : Le projet est-il open source ?

R : Pour l'instant non, c'est un projet PFE. Mais l'architecture est conforme aux standards open source. Si la FSBM le souhaite, on peut publier.

Chapitre 8 - Q/R pieges

Les jurys aiment poser des questions **pieges** ou **ouvertes**. Voici comment y répondre intelligemment.

Q : Pourquoi pas ChatGPT directement ?

R : ChatGPT n'a pas le contexte FSBM. Il inventerait les infos. Il faut soit le fine-tuner (cher, complexe), soit faire du RAG. Notre architecture RAG +TF-IDF donne 100% d'infos fiables + 100% de disponibilité (fallback).

Q : C'est juste du chatbot, qu'y a-t-il d'innovant ?

R : Trois innovations : (1) Cascade fallback Groq -> HF -> TF-IDF qui garantit 100% uptime, (2) Détection darija avec 461 patterns et algorithme numérique (rare), (3) Personnalisation genrée pour la darija (khoya/khti/lalla).

Q : Vous avez quoi de plus qu'un FAQ statique ?

R : Un FAQ statique force l'utilisateur à chercher. Le bot COMPREND une question formulée naturellement, dans 3 langues, et trouve la réponse pertinente. C'est un saut UX majeur. + mémoire conversationnelle pour multi-tours.

Q : Et si le dataset est biaisé ?

R : Bon point. On documente les biais : 461 patterns darija mais 188 en FR. On essaie de balancer. Et le feedback user permet de détecter les angles morts. Pas parfait, mais conscient.

Q : Le LLM peut-il dire des choses inappropriées ?

R : Risque limité par : (1) RAG impose le contexte FSBM, (2) System prompt strict interdit politique/religion, (3) Garde-fous Groq sur safety. Pas zéro risque mais contrôle. En production : modération manuelle prévue.

Q : Combien coûte le projet en production ?

R : Hosting + DB : ~20 EUR/mois (VPS). Groq API : gratuit jusqu'à 30000 tokens/min. Largement suffisant. Domaine + SSL : ~50 EUR/an. Total : moins de 300 EUR/an.

Q : Et si Groq devient payant ou ferme ?

R : Architecture découplée. On switch vers HuggingFace, OpenAI, ou un LLM local (Ollama). Le code LLMService est conçu pour ça : juste changer le client.

Q : Pourquoi 28 intents et pas 100 ?

R : Principe 80/20 : 28 intents couvrent 80% des questions. Au-delà, on a explosion combinatoire et confusion entre intents trop proches. Le LLM (RAG) prend le relais pour les 20% restants.

Q : Pourquoi n'avez-vous pas fait de mobile app ?

R : Scope du PFE limite a 3 mois. On a privilegie l'architecture solide + multilingue + LLM. Une mobile app est juste un client de plus sur la meme API REST. C'est Phase 3.

Q : Quelle est votre contribution personnelle (vs frameworks) ?

R : Frameworks = outils (Angular, FastAPI). Notre travail : (1) Architecture microservices specifique, (2) Dataset trilingue de 835 patterns, (3) Algorithme de detection darija original, (4) Logique RAG avec TF-IDF retriever, (5) Cascade fallback, (6) Personnalisation genre/nom.

Chapitre 9 - Backup plan

9.1 Loi de Murphy : tout peut casser le jour J

Anticiper les pires scenarios. Voici les plans B/C/D.

9.2 Si pas de WiFi / pas d'internet

- + Hotspot 4G sur ton telephone (vraiment)
- + **Tout doit tourner LOCAL** : MySQL local, MongoDB local
- + Mode TF-IDF (sans Groq) doit fonctionner 100%
- + Avoir desactive Groq par default dans .env : **GROQ_API_KEY vide**
- + Au pire : video pre-enregistree de demo

9.3 Si le PC plante

- + **Backup PDF** des slides sur cle USB
- + **Backup PDF** des captures d'ecran de demo
- + Tablette en backup avec slides
- + Pouvoir presenter SANS demo : juste les slides + capture

9.4 Si un service crash en demo

- + Garder un terminal pre-lance pret a relancer
- + Ne pas paniquer, dire : 'Un instant, je relance le service'
- + Pendant le restart, EXPLIQUER ce qui se passe (architecture, ports, etc.)
- + Si ca ne repart pas : passer aux captures d'ecran
- + Si vraiment ca refuse : 'En conditions reelles, ce service tournerait sur un serveur dedie avec auto-restart. Voici les captures...'

9.5 Si Groq API down

- + **Pas grave** : la cascade fallback prend le relai (TF-IDF)
- + Phrase : 'Vous remarquez le badge *provider: tfidf* : c'est notre fallback automatique qui s'active si Groq est indisponible. La reponse est moins fluide mais 100% fiable.'
- + C'est en fait un AVANTAGE a demontrer : la robustesse de l'archi

9.6 Si question dont tu n'as pas la reponse

+ **Surtout ne pas mentir**

+ 'Excellente question, je n'ai pas la reponse precise. Mon hypothese serait... mais je verifierai apres la soutenance.'

+ Tu peux dire : 'C'est dans les perspectives de Phase 2 / 3'

+ Le jury VALORISE l'honnetete intellectuelle

[★] RAPPEL : les jurys ne testent pas si tu sais TOUT. Ils testent si tu sais REFLECHIR. Une question sans reponse parfaite est une opportunit   de montrer ta capacite a structurer ta pensee.

Chapitre 10 - Erreurs a eviter

10.1 Erreurs de presentation

- **Lire les slides** mot pour mot (le jury sait lire)
- Dire 'tu' au lieu de 'vous'
- Parler trop vite par stress
- Tourner le dos au jury pour regarder l'ecran
- Critiquer Angular/React/Python (rester neutre)
- Promettre des choses non realisees
- Sous-vendre ton travail ('c'est rien de special')
- Sur-vendre ton travail (mentir / exagerer)

10.2 Erreurs techniques

- **Faire la demo sans avoir verifie 5 min avant**
- Avoir des warnings/errors visibles en console
- Coder en live (sauf si tres prepare)
- Faire 'rm -rf' devant le jury (le stress fait des betises)
- Avoir le mot de passe d'admin visible
- Avoir l'API key Groq visible a l'ecran (CRITIQUE)
- Ne pas connaitre une partie de TON code

10.3 Erreurs de fond

- Ne pas savoir expliquer une ligne de TON code
- Avoir copie-colle StackOverflow sans comprendre
- Dire que tu as fait quelque chose si l'autre membre l'a fait
- Eviter une question (toujours mieux d'admettre)
- Comparer ton projet a Google / OpenAI (humilite)
- Pretendre que c'est innovant alors que c'est standard

Chapitre 11 - Conclusion

11.1 Phrases de cloture

Termine sur une note positive et ouverte :

'En conclusion, FSBM Platform repond aux 4 enjeux poses : disponibilite 24/7, support multilingue, comprehension du langage naturel, et scalabilite. Le systeme est deja fonctionnel et pret pour une phase pilote. Les perspectives incluent l'authentification etudiants, l'integration WhatsApp, et une app mobile. Nous sommes a votre disposition pour vos questions. Merci de votre attention.'

11.2 Pendant les questions

- + Ecouter **jusqu'au bout** avant de repondre
- + Reformuler la question : 'Si je comprends bien, vous demandez...'
- + Repondre directement, sans broder
- + Si question complexe : 'Je vais structurer ma reponse en 3 points'
- + Si tu ne sais pas : reconnaitre + proposer une piste
- + Remercier : 'Merci pour cette question'

11.3 Apres la soutenance

- Remercier le jury, sourire, poignee de main
- Quitter avec ton support, ne rien laisser sur l'ecran
- Profite du moment, tu as termine !
- Quel que soit le resultat : tu as appris enormement

11.4 Recap des 10 PDFs

- PDF 01 - Architecture Globale
- PDF 02 - Frontend Angular Complet
- PDF 03 - Backend FastAPI Complet
- PDF 04 - MySQL + SQLAlchemy Complet
- PDF 05 - MongoDB + NoSQL Complet
- PDF 06 - NLP + IA Chatbot Complet
- PDF 07 - APIs + Communication
- PDF 08 - Securite Complete
- PDF 09 - Workflow Complet

- PDF 10 - Guide Soutenance (ce PDF)

[✓] Bonne chance pour ta soutenance ! Tu as fait un travail solide. Le jury verra ton travail, ta rigueur, ta capacite a expliquer. Reste calme, sois honnete, et passe un bon moment. Tu vas reussir.